

LocalPass

Local Password Manager

Software Design Document

Version	1.0.0
Status	Draft
Author	Mahdi Mansour
Date	May 2026
Stack	C++17 · OpenSSL · CMake

1. Overview

LocalPass is a fully offline, command-line password manager written in C++17. It stores all credentials in a single encrypted vault file on disk, never transmitting data over a network. The vault is protected by a master password that is never stored — only derived into an encryption key at runtime using a modern key-derivation function.

This document covers the system architecture, cryptographic design, data model, CLI interface, security considerations, and development roadmap.

Property	Value
Language	C++17
Build system	CMake 3.20+
Crypto library	OpenSSL 3.x
Serialization	nlohmann/json
Target platforms	Linux, macOS, Windows
License	MIT

2. Goals & Non-Goals

Goals

- Store credentials (site, username, password, notes) in an encrypted local vault
- Protect the vault with AES-256-GCM authenticated encryption
- Derive the encryption key from a master password using Argon2id — never store the master password
- Generate strong, cryptographically random passwords of configurable length and character sets
- Copy passwords to the clipboard with automatic clearing after 30 seconds
- Wipe sensitive data from memory after use to reduce in-process exposure
- Provide a clean CLI usable in scripts and terminal workflows

Non-Goals

- Cloud sync or remote backup (out of scope for v1)
- A graphical user interface
- Browser auto-fill integration (stretch goal only)
- Multi-user or shared vault support
- Importing from other password managers (stretch goal)

3. System Architecture

LocalPass is structured as five distinct layers. Each layer has a single responsibility, communicating only with the layer directly above or below it.

Layer	Module	Responsibility
CLI	cli/commands.cpp	Parse arguments, route to vault or generator, handle user prompts
Auth	crypto/kdf.cpp	Derive 256-bit key from master password + salt via Argon2id
Vault	vault/vault.cpp	CRUD operations on entries; serialize/deserialize to JSON
Crypto	crypto/aes.cpp	AES-256-GCM encrypt / decrypt of the serialized JSON blob
Storage	vault/entry.cpp	Read/write the .vault binary file; manage salt and IV

Directory Layout

```
password-manager/  
├── src/  
│   ├── main.cpp  
│   ├── cli/commands.cpp  
│   ├── crypto/kdf.cpp  
│   ├── crypto/aes.cpp  
│   ├── vault/vault.cpp  
│   ├── vault/entry.cpp  
│   └── utils/  
│       ├── generator.cpp  
│       └── clipboard.cpp  
├── tests/  
├── CMakeLists.txt  
└── README.md
```

4. Cryptographic Design



Security is the core invariant of this project. Every cryptographic decision below is deliberate and documented.

4.1 Key Derivation — Argon2id

The master password is **never stored** — not even hashed. Instead, it is fed into Argon2id (RFC 9106) each time the vault is unlocked to derive a 256-bit symmetric key. Argon2id is the winner of the Password Hashing Competition and is resistant to both GPU-parallel and side-channel attacks.

Parameter	Value	Rationale
Algorithm	Argon2id	Hybrid — resists GPU attacks (data-dependent) and side-channels (data-independent)
Memory cost	65536 KB (64 MB)	Increases cost for parallel brute-force attempts
Iterations	3	OWASP minimum recommendation for Argon2id
Parallelism	4 threads	Matches typical desktop CPU core count
Salt length	32 bytes	Randomly generated per vault; stored in vault header
Output length	32 bytes	256 bits — direct input to AES-256-GCM

4.2 Encryption — AES-256-GCM

The derived key encrypts the entire JSON payload using AES-256-GCM. GCM mode provides authenticated encryption — it both encrypts the data and produces an authentication tag that detects any tampering with the ciphertext.

- A fresh 12-byte IV (nonce) is generated using `RAND_bytes()` on every save
- The GCM authentication tag (16 bytes) is verified before any decryption output is returned
- If tag verification fails, the function returns an error — no partial plaintext is ever exposed

✓ The vault file is 100% opaque without the master password. Even knowing the exact algorithms and parameters, an attacker cannot decrypt or tamper with the vault.

4.3 Vault File Format

The `.vault` file is a flat binary with a fixed-size header followed by the ciphertext blob.

Offset	Length	Content
0	4 bytes	Magic bytes: 0x4C 0x50 0x41 0x53 (LPAS)
4	2 bytes	File format version (uint16, little-endian)
6	32 bytes	Argon2id salt (random, per vault)

38	12 bytes	AES-GCM nonce / IV (random, per save)
50	16 bytes	GCM authentication tag
66	variable	AES-256-GCM ciphertext (encrypted JSON)

4.4 Secure Memory Handling

Decrypted plain text is kept in memory only as long as needed and then wiped using platform-safe routines that the compiler cannot optimize away.

- Use `OPENSSL_cleanse()` — not `memset()` — to zero sensitive buffers
- Passwords are stored in a custom `SecureString` wrapper that zeroes memory on destruction
- Master password input uses `getpass()` (Unix) or `ReadConsole()` with echo disabled (Windows)
- The decrypted JSON blob is freed immediately after entries are parsed into memory

5. Data Model

Inside the encrypted vault, entries are stored as a JSON array. Each entry is a flat object with the following fields:

Field	Type	Description
id	string (UUID v4)	Unique identifier for the entry
site	string	Website or service name (e.g. github)
username	string	Login username or email address
password	string	The credential (stored encrypted)
notes	string (optional)	Free-text notes
created_at	ISO 8601 timestamp	Entry creation time (UTC)
updated_at	ISO 8601 timestamp	Last modification time (UTC)

Example decrypted JSON payload:

```
[
  {
    "id": "a3f1c2d4-...",
    "site": "github",
    "username": "your@email.com",
    "password": "T7#kQw...Xm9",
    "notes": ""
  }
]
```

```
"created_at": "2026-05-01T10:00:00Z",
"updated_at": "2026-05-01T10:00:00Z"
}
]
```

6. CLI Interface

All commands follow the pattern:

```
pm <command> [options]
```

Command	Description	Example
pm init	Create a new encrypted vault	pm init
pm add <site>	Add a new credential entry	pm add github
pm get <site>	Decrypt and copy password to clipboard	pm get github
pm list	List all stored site names	pm list
pm delete <site>	Remove an entry from the vault	pm delete github
pm generate	Generate and optionally store a password	pm generate --length 24 --symbols
pm update <site>	Update an existing entry	pm update github

⚠ pm get always copies the password to clipboard and clears it after 30 seconds. The password is never printed to stdout to avoid shell history or log exposure.

Password Generator Options

Flag	Default	Description
--length N	16	Length of generated password
--symbols	off	Include special characters (!@#\$...)
--no-upper	off	Exclude uppercase letters
--no-digits	off	Exclude digits
--save	off	Save generated password to vault entry

7. Security Considerations

Threat Model

LocalPass is designed to protect against the following threat scenarios:

- **Stolen vault file:** an attacker who copies the .vault file gains nothing without the master password. The combination of Argon2id and AES-256-GCM makes brute-force computationally infeasible.
- **Memory inspection:** sensitive strings are zeroed immediately after use. An attacker with memory-read access gets a narrow window.
- **Shoulder surfing / shell history:** passwords are never printed to stdout. Master password input disables terminal echo.
- **Vault tampering:** GCM authentication detects any modification to the ciphertext — even a single flipped bit causes decryption to fail.

Out-of-Scope Threats

- ✕ LocalPass does not protect against a compromised operating system, a keylogger active at the time of master password entry, or physical access while the vault is unlocked. These are outside the scope of any local password manager.

Key Security Properties

Property	Implementation	Status
Authenticated encryption	AES-256-GCM with 16-byte tag	✓ Implemented
Memory-hard KDF	Argon2id (64MB / 3 iter / 4 thread)	✓ Implemented
Secure random generation	OpenSSL RAND_bytes() + /dev/urandom	✓ Implemented
Secure memory wipe	OPENSSL_cleanse() on all secrets	✓ Implemented
No master password storage	KDF derives key; password discarded	✓ By design
Clipboard auto-clear	30-second timer on clipboard write	✓ Implemented

8. Libraries & Dependencies

Library	Version	Purpose
---------	---------	---------

OpenSSL	3.x	AES-256-GCM, Argon2id, RAND_bytes, OPENSSL_cleanse
nlohmann/json	3.11+	JSON serialization of vault entries (header-only)
CMake	3.20+	Cross-platform build configuration
GoogleTest	1.14+	Unit and integration testing

All dependencies are open-source with permissive licenses (OpenSSL License, MIT). They are pinned to specific versions via CMake FetchContent to ensure reproducible builds.

9. Testing Strategy

Unit Tests

- KDF — known-answer tests against reference Argon2id vectors
- AES-GCM — encrypt then decrypt round-trip; tamper detection (flipped byte → failure)
- Password generator — character set coverage, length correctness, CSPRNG statistical tests
- Vault CRUD — add, get, update, delete on an in-memory vault

Integration Tests

- `pm init` → `pm add` → `pm get` full round-trip on a temp file
- Vault file persists correctly across process restarts
- Wrong master password produces decryption failure with no partial data

Security-Specific Tests

- Memory wipe: verify buffer contains zeros after SecureString destruction (via custom allocator hook)
- Vault tamper: modify one byte of ciphertext and verify decryption returns an error
- Salt uniqueness: two `pm init` calls produce different salts

10. Development Roadmap

Week	Milestone	Deliverable
Week 1	Foundation	CMake setup, vault file format, AES-GCM + Argon2id working end-to-end

Week 2	Core CLI	All CRUD commands, SecureString, master password prompt
Week 3	Generator + UX	Password generator, clipboard handler, pm list, pm update
Week 4	Quality	Full test suite, README, memory wipe verification, cross-platform build
Stretch	TOTP	RFC 6238 TOTP generator for 2FA codes
Stretch	TUI	ncurses / ftxui terminal UI for vault browsing

11. Open Questions

- **Vault backup strategy:** Should pm init automatically create a recovery sheet with vault metadata? Or is that a v2 concern?
- **Windows clipboard:** The clipboard auto-clear implementation differs significantly between X11, Wayland, and Win32 — which platforms are must-haves for v1?
- **Entry search:** Should pm get support fuzzy matching (e.g. pm get git → matches github)? Or should it require an exact site key?
- **Master password change:** pm rekey should re-derive the key and re-encrypt all entries. Design needed before vault format is finalized.

Appendix A — References

- RFC 9106 — Argon2 Memory-Hard Function: <https://www.rfc-editor.org/rfc/rfc9106>
- NIST SP 800-38D — AES-GCM specification: <https://csrc.nist.gov/publications/detail/sp/800-38d/final>
- OWASP Password Storage Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
- OpenSSL EVP documentation: https://www.openssl.org/docs/man3.0/man3/EVP_EncryptInit.html

Appendix B — Glossary

Term	Definition
AES-GCM	Advanced Encryption Standard in Galois/Counter Mode — provides confidentiality and integrity
Argon2id	Memory-hard password hashing algorithm used for key derivation
CSPRNG	Cryptographically Secure Pseudo-Random Number Generator
GCM auth tag	16-byte integrity checksum produced by AES-GCM; detects any tampering

IV / Nonce	Initialization Vector — a random value that ensures ciphertext uniqueness
KDF	Key Derivation Function — derives a fixed-size key from a variable-length password
TOTP	Time-based One-Time Password (RFC 6238) — used for 2FA codes